

# AUTHENTICATION HANDBOOK

## ≡ (JWT + OAUTH 2.0) ≡

"Secure today,  
access tomorrow." ✨

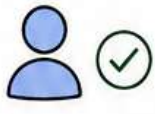
PAGE 1

## 1. AUTHENTICATION FUNDAMENTALS

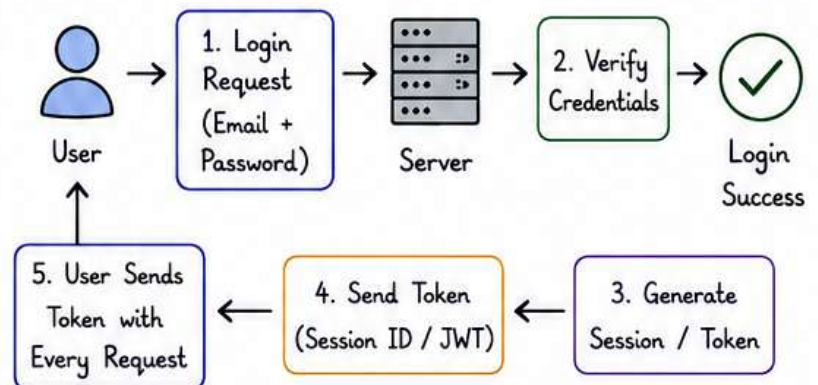
Authentication is about verifying who you are.

Authorization is about what you are allowed to do.

### 1.1 Authentication vs Authorization

| Authentication                                                                    | Authorization                                                                     |
|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
|  |  |
| Verifies the identity of a user.                                                  | Determines what a user can access.                                                |
| Example: Login with email & password.                                             | Example: Accessing admin dashboard.                                               |

### 1.2 Basic Login Flow



### 1.3 Sessions vs Tokens

|                      | Session-Based (Authentication) | Token-Based (JWT)              |
|----------------------|--------------------------------|--------------------------------|
| Where data is stored | Server                         | Client (Token)                 |
| Stored on            | Cookie (Session ID)            | LocalStorage / Cookie / Memory |
| Scalability          | Harder (Need session store)    | Easier (Stateless)             |
| Revocation           | Easy (Invalidate session)      | Harder (Need blacklist)        |
| Best for             | Traditional Web Apps           | APIs, SPAs, Mobile Apps        |

### 1.4 Cookies (Quick Overview)

- Small pieces of data stored in browser.
- Sent automatically with every request to the domain.
- Can store Session ID or JWT.
- Important Flags:

**HttpOnly** → Not accessible via JavaScript (Protects from XSS)

**Secure** → Sent only over HTTPS

**SameSite** → Controls CSRF attacks (Lax / Strict / None)

### 1.5 Common HTTP Status Codes

401

#### Unauthorized

Authentication is required or failed.  
(Invalid / missing token or credentials)

403

#### Forbidden

Authenticated, but not allowed to access the resource.

### 1.6 Key Takeaway



Authentication answers the question:  
"WHO ARE YOU?"

Authorization answers the question:  
"WHAT CAN YOU DO?"



**TIP:** Always authenticate first, then authorize.  
You can't authorize an anonymous user.

IG: abhi\_techhub

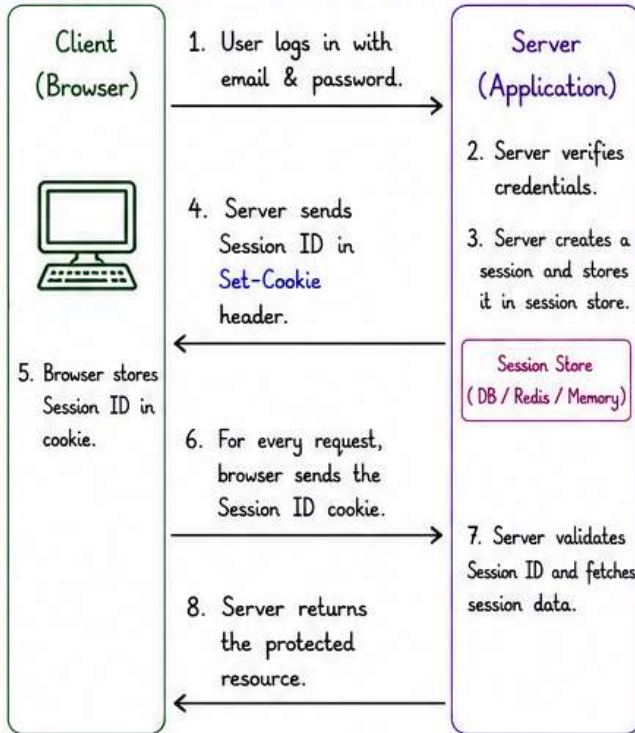


## 2. SESSION-BASED AUTHENTICATION

PAGE 2

In session-based auth, the server stores user session information.  
The client only holds a **Session ID** (in cookie).

### 2.1 How It Works (Flow)



### 2.2 Example: Request with Session ID

**Request:**

GET /dashboard HTTP/1.1

Host: www.example.com

Cookie: sessionId = abc123xyz

**Server:**

1. Reads sessionId from cookie.
2. Finds abc123xyz in session store.
3. If valid → allow access.  
If not valid / expired → 401 Unauthorized.

Only the  
Session ID  
travels, not  
the data!

### 2.3 Session Store (Where to Store?)

- Database → Easy but slower
- Redis → Fast, scalable, widely used
- Memcached → Very fast (key-value store)
- In-Memory → Good for small apps (not scalable)
- File / Others → Not recommended for production

### 2.4 Pros and Cons

#### Advantages

- ✓ Simple to implement
- ✓ Easy to revoke / invalidate session
- ✓ More secure (Session ID is small)
- ✓ Works well with traditional web apps
- ✓ Can store more data on server

#### Disadvantages

- ✗ Not stateless (needs server storage)
- ✗ Harder to scale (requires shared store)
- ✗ Extra load on server / store
- ✗ Not ideal for mobile / APIs
- ✗ More server memory usage

### 2.5 When to Use Sessions?

- Traditional Web Applications
- When you need to logout from all devices
- When immediate session invalidation is required
- When sensitive data should not travel to client
- When you can maintain a server-side store

### 2.6 Session vs Token (Quick Recap)

| Feature               | Session-Based               | Token-Based (JWT)             |
|-----------------------|-----------------------------|-------------------------------|
| Where data is stored  | Server (Session Store)      | Client (Token)                |
| What the client holds | Session ID (in Cookie)      | JWT (Full token)              |
| Scalability           | Harder (needs shared store) | Easier (Stateless)            |
| Revocation            | Easy (Delete session)       | Harder (Blacklist / Expiry)   |
| Best for              | Web Apps                    | APIs, SPAs, Mobile Apps       |
| Data in transit       | Small (Session ID)          | Large (Token contains claims) |

**Remember:**

**Sessions** keep data on the server.  
**Tokens** keep data on the client.



Server  
Keeps Data

VS



Client  
Keeps Data



**TIP:**

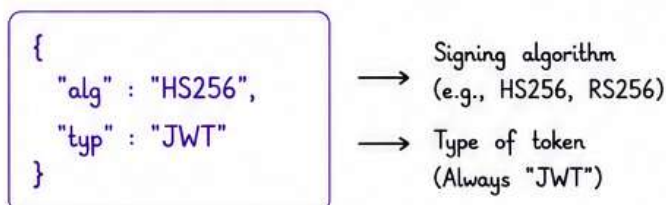
Sessions are great for server-rendered apps.  
Tokens are great for APIs and modern client apps.  
Choose based on your use case!

IG: abhi\_techhub



PAGE 3

### 3.2 Header (Example)



```
{
  "sub" : "123",
  "name" : "Abhi Kumar",
  "email" : "abhi@example.com",
  "role" : "admin",
  "iat" : 1715496000,
  "exp" : 1715499600
}
```

- Subject (User ID)
- User Name
- User Email
- User Role
- Issued At (Time)
- Expires At (Time)

### 3.4 Signature

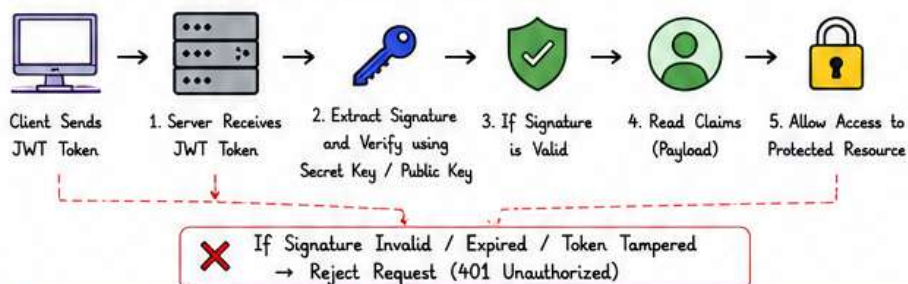
Server uses the secret key (or public key) to verify the signature.  
If signature is valid  $\rightarrow$  token is trusted.

### 3.5 Encoding vs Encryption

| Aspect            | Encoding (Base64)                 | Encryption              |
|-------------------|-----------------------------------|-------------------------|
| Purpose           | Make data readable in JSON format | Hide data (keep secret) |
| Can be decoded?   | Yes (Anyone)                      | No (Only with key)      |
| JWT uses?         | Encoding                          | Not Encryption          |
| Safe for secrets? | No                                | Yes                     |
| Example           | Base64UrlEncode                   | AES, RSA, etc.          |

```
graph LR; A[1. Create Header] --> B[2. Create Payload (Claims)]; B --> C[3. Encode Header & Payload]; C --> D[4. Sign with Secret Key (or Private Key)]; D --> E[5. Final JWT Token]
```

### 3.7 JWT Verification Flow (On Server)



### 3.8 Key Points

- ✓ JWT is stateless.
- ✓ Server does not store session (only verifies token).
- ✓ Token is self-contained.
- ✓ Do not store sensitive data in payload.
- ✓ Always use HTTPS.
- ✓ Keep secret keys safe.
- ✓ Short expiry + refresh token is best practice.



**TIP:** JWT is not encrypted, it is encoded.  
Anyone can read the payload, so never put passwords, credit card numbers or other sensitive info in JWT.

IG: [abhi\\_techhub](#)

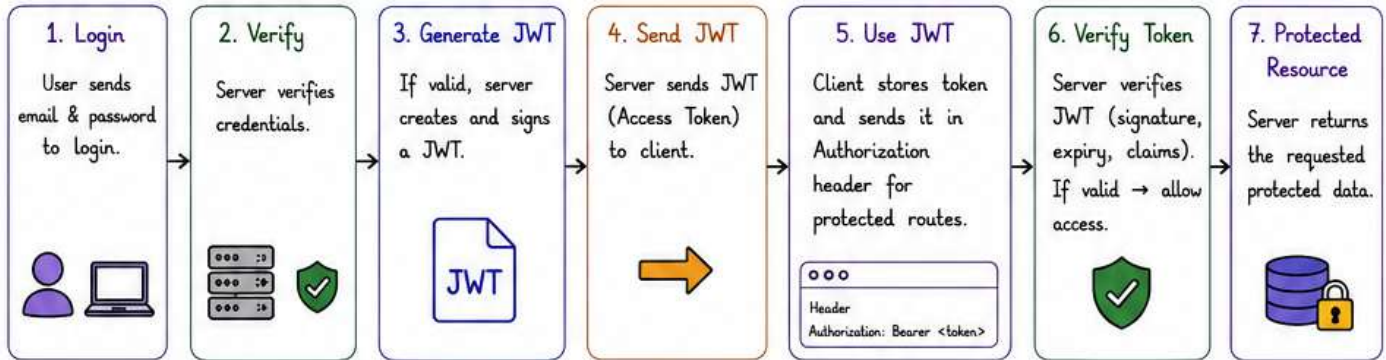


### 4. JWT AUTHENTICATION FLOW

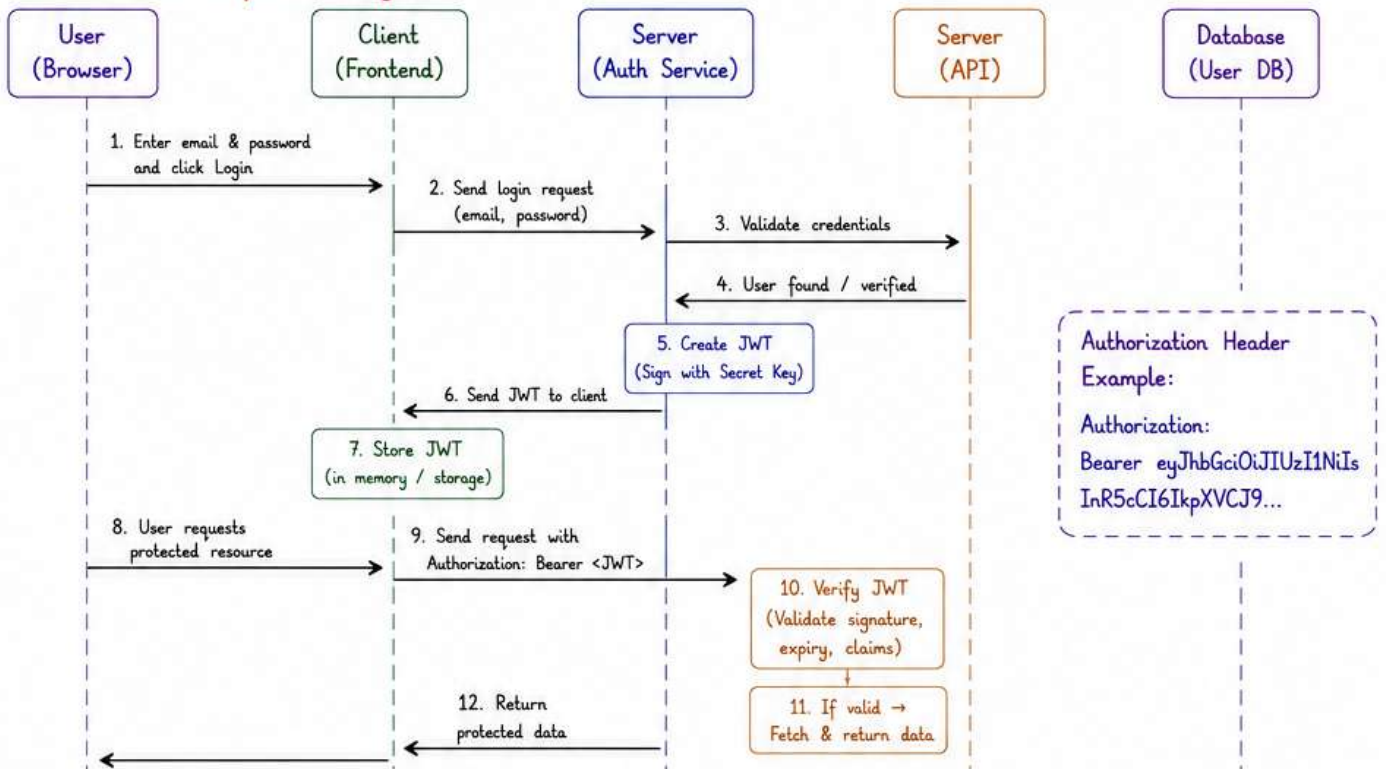
PAGE 4

JWT is stateless. Once issued, the token is used for every request until it expires.

#### 4.1 End-to-End Flow (High Level)



#### 4.2 Detailed Sequence Diagram



#### 4.3 What Happens in Each Step?

| Step | What Happens                                             | Who Does It? |
|------|----------------------------------------------------------|--------------|
| 1    | User submits login form.                                 | User         |
| 2    | Server checks email & password in DB.                    | Auth Service |
| 3    | If valid, server creates JWT (header.payload.signature). | Auth Service |
| 4    | JWT is sent back to the client.                          | Auth Service |
| 5    | Client stores JWT and sends in Authorization header.     | Client       |
| 6    | Server (API) verifies JWT on every request.              | API Server   |
| 7    | If valid → access granted and data returned.             | API Server   |

#### 4.4 Notes

- JWT is stateless.
- No session store needed on server.
- Server only verifies the token.
- Token contains all required claims.
- If token is expired / invalid → 401 Unauthorized.
- If token is valid but user not allowed → 403 Forbidden.



**TIP:** With JWT, server is "trusting" the token. So, protect your secret key and token expiration settings carefully.

#### Remember:

Short lived Access Token  
+ Refresh Token = Best Practice

IG: abhi\_techhub ❤️

## 5. ACCESS TOKEN + REFRESH TOKEN

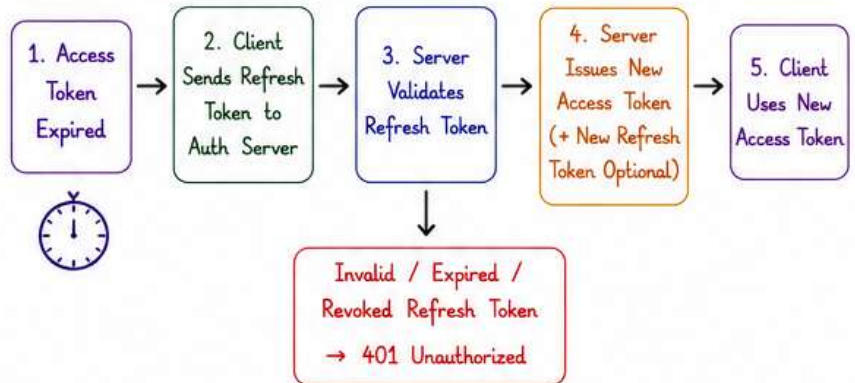
PAGE 5

Using two tokens improves security and user experience.

### 5.1 Why Two Tokens?

- **Access Token** → Short lived (5 - 15 min)  
Used to access APIs.
- **Refresh Token** → Long lived (Days / Weeks)  
Used only to get new access token.
- Even if Access Token is leaked,  
damage is limited (short expiry).  
Refresh Token stays secure (HttpOnly cookie)..

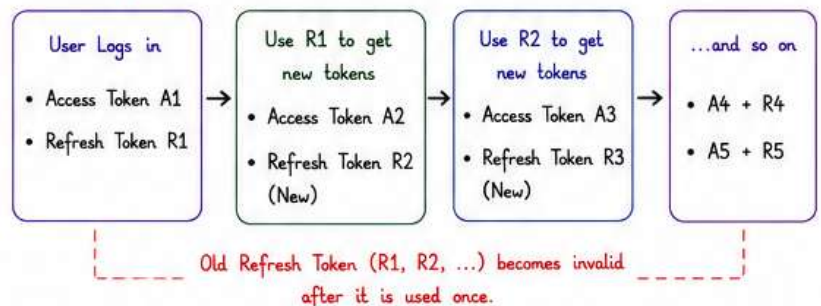
### 5.2 Refresh Flow



### 5.3 Token Expiry Strategy (Example)

| Token                  | Expiry             |
|------------------------|--------------------|
| Access Token           | 5 - 15 minutes     |
| Refresh Token          | 7 - 30 days        |
| Refresh Token Rotation | Yes (Recommended)  |
| Reuse Detection        | Yes (For Security) |

### 5.4 Refresh Token Rotation (Recommended)



### 5.5 Reuse Detection (Important)

- If an old Refresh Token is used again,  
it means token is stolen.
- In this case:
  - ① Invalidate that Refresh Token.
  - ② Invalidate all tokens issued after it.
  - ③ Ask user to login again.
- Protects from token theft / replay attack.



### 5.6 Logout & Revocation

| User Logout                                                                                                                                                                            | Admin / Server Revocation                                                                                                                                                                                     |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>• Delete Refresh Token from server store.</li> <li>• Client deletes Access Token from storage.</li> <li>• User needs to login again.</li> </ul> | <ul style="list-style-type: none"> <li>• Revoke Refresh Token (for suspicious activity).</li> <li>• Blacklist Access Token (jti) if required.</li> <li>• Force logout from all devices (optional).</li> </ul> |

### 5.7 Access Token vs Refresh Token

| Feature        | Access Token                           | Refresh Token                        |
|----------------|----------------------------------------|--------------------------------------|
| Purpose        | Access protected resources (APIs)      | Get new access token                 |
| Lifetime       | Short (5 - 15 min)                     | Long (Days / Weeks)                  |
| Sent In        | Authorization header (Bearer Token)    | HttpOnly Cookie or Secure Storage    |
| Risk if Leaked | Limited (short expiry)                 | High (can issue new access token)    |
| Storage        | Memory / LocalStorage (or Header only) | HttpOnly Secure Cookie (Recommended) |
| Rotation       | No (New one issued often)              | Yes (Rotate on every use)            |
| Revocation     | Blacklist (jti) if needed              | Remove from store → Cannot be used   |
| Used For       | Every API request                      | Only when access token expires       |

#### Cookie (For Refresh Token)

- Set-Cookie: refreshToken=xxx;  
HttpOnly; Secure;  
SameSite=Strict; Path=/;  
Max-Age=604800  
(7 days)
- HttpOnly → Not accessible via JavaScript (XSS safe)
- Secure → Sent only over HTTPS
- SameSite → Protects from CSRF (Strict / Lax / None)



**TIP:** Short lived Access Token + Long lived Refresh Token  
= Secure, Scalable & Smooth User Experience.



## 6. WHERE SHOULD YOU STORE JWT?

PAGE 6

Storing JWT securely is critical. Each option has trade-offs.

### 6.1 Storage Options Comparison

| Storage Option                     | Where it is Stored               | Accessible By JavaScript? | XSS Risk                 | CSRF Risk                   | Persistence            | Best For                                                                                                     |
|------------------------------------|----------------------------------|---------------------------|--------------------------|-----------------------------|------------------------|--------------------------------------------------------------------------------------------------------------|
| 1. HttpOnly Cookie                 | Browser Cookie (HttpOnly = true) | No (Not accessible)       | Low (Protected)          | Yes (If no CSRF protection) | Configurable (Expires) | <ul style="list-style-type: none"> <li>Web Apps</li> <li>Most secure for tokens</li> </ul>                   |
| 2. Memory (React State / Variable) | Browser Memory (Volatile)        | Yes (Accessible)          | High (Vulnerable to XSS) | Low (Safe)                  | None (Lost on refresh) | <ul style="list-style-type: none"> <li>SPA Apps</li> <li>Short lived sessions</li> </ul>                     |
| 3. LocalStorage                    | LocalStorage (Persistent)        | Yes (Accessible)          | High (Vulnerable to XSS) | Low (Safe)                  | Long Term (Persists)   | <ul style="list-style-type: none"> <li>Not Recommended for JWT</li> <li>High XSS Risk</li> </ul>             |
| 4. SessionStorage                  | SessionStorage (Tab based)       | Yes (Accessible)          | High (Vulnerable to XSS) | Low (Safe)                  | Until Tab Closes       | <ul style="list-style-type: none"> <li>Slightly better than LocalStorage</li> <li>Still not ideal</li> </ul> |

### 6.2 Recommendation

- ✓ Best Practice: Store JWT (Refresh Token) in HttpOnly, Secure, SameSite cookies.
- ✓ Store Access Token in memory (not localStorage) for SPAs if possible.
- ✓ Always use HTTPS.

### 6.3 Important Cookie Flags

- HttpOnly → Not accessible via JavaScript (Prevents XSS theft)
- Secure → Sent only over HTTPS
- SameSite → Controls CSRF attacks (Strict / Lax / None)
- Path → Limits where cookie is sent
- Max-Age → Defines cookie lifetime (in seconds)

### 6.4 Common Security Mistakes

- ✗ Storing JWT in localStorage.
- ✗ Not using HttpOnly cookies for refresh token.
- ✗ Not using HTTPS (token can be sniffed).
- ✗ Not validating token expiration.
- ✗ Not using SameSite cookies (CSRF risk).

### 6.5 Quick Decision Guide

#### Use HttpOnly Cookie

- You want long lived refresh token
- You want better protection from XSS
- You want cookie-based auth (traditional web apps)

#### Use Memory (Not localStorage)

- You want short lived access token
- You are building an SPA
- You want to reduce XSS risk



**TIP:** Security is all about reducing the impact of token theft.  
Use the safest storage + shortest token lifecycle.

IG: abhi\_techhub



## 7. OAUTH 2.0 FUNDAMENTALS

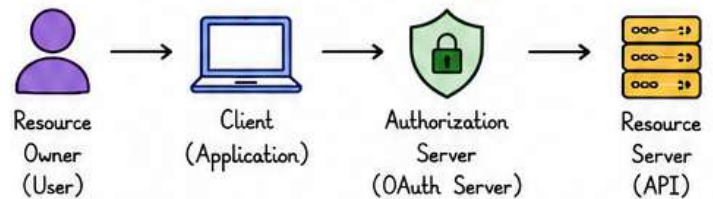
PAGE 7

OAuth 2.0 is an **authorization** framework, not an authentication protocol. It allows a user to grant limited access to their resources without sharing credentials.

### 7.1 Key Roles in OAuth 2.0

|                                                                                     |                                 |                                                                                |
|-------------------------------------------------------------------------------------|---------------------------------|--------------------------------------------------------------------------------|
|    | 1. Resource Owner (End User)    | The user who owns the data and grants permission.                              |
|    | 2. Client (Application)         | The application that requests access to user's resources.                      |
|    | 3. Authorization Server (AS)    | The server that authenticates the user and issues access tokens to the client. |
|  | 4. Resource Server (API Server) | The server that hosts the user's protected resources.                          |

### 7.2 How OAuth 2.0 Works (High Level)



1. User tries to access a resource in Client.
2. Client redirects user to Authorization Server to login and grant permission (with scopes).
3. User authenticates and consents.
4. Authorization Server issues an **Access Token** to the Client.
5. Client uses the **Access Token** to access the **Resource Server**.
6. **Resource Server** validates the token and returns the requested data.

### 7.3 Important Concepts

- **Scope** → Defines the level of access requested. (e.g., read:profile, write:email)
- **Consent** → User gives permission to the client.
- **Access Token** → Short lived token used to access APIs.
- **Refresh Token** → (Optional) Used to get new access token when the old one expires.
- **Grant Type** → The flow used to obtain the access token.

### 7.4 OAuth 2.0 is NOT Authentication

OAuth 2.0 = Authorization (Delegation)

- It allows a client to access resources on behalf of the user with user's permission.

Authentication = Proving Identity

- It verifies who the user is (e.g., using username and password).

Example: You can use OAuth 2.0 with Google to allow your app to read Google Drive files without knowing your password.

### 7.5 Example: Real World Analogy



You (user) allow the Notes App to access your Google Drive files through Google (OAuth Server) without sharing your Google password.

### 7.6 OAuth 2.0 vs Other Methods

| Feature              | OAuth 2.0                                  | JWT                                      | Session (Server-Side)                  |
|----------------------|--------------------------------------------|------------------------------------------|----------------------------------------|
| Purpose              | Authorization (Delegated Access)           | Authentication (Token based)             | Authentication (Server stores session) |
| Stores User Session? | No                                         | No                                       | Yes (on server)                        |
| Token Type           | Access Token (+ Refresh Token)             | Access Token (JWT)                       | Session ID (in Cookie)                 |
| Stateless?           | Yes                                        | Yes                                      | No                                     |
| Main Use             | Allow third-party apps to access user data | Secure API access for authenticated user | Web apps with server managed sessions  |
| Example              | Login with Google, Facebook                | Access your own API                      | E-commerce website (cart, login)       |



TIP:

- ✓ OAuth 2.0 = Get permission (Authorization)
- ✓ JWT = Prove who you are (Authentication)
- ✓ Don't confuse the two!

IG: abhi\_techhub

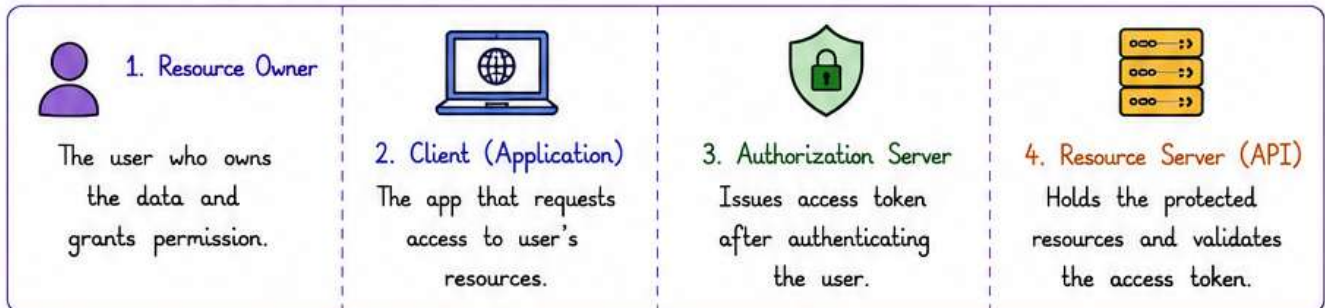


## 8. OAUTH 2.0 AUTHORIZATION CODE FLOW (WITH PKCE)

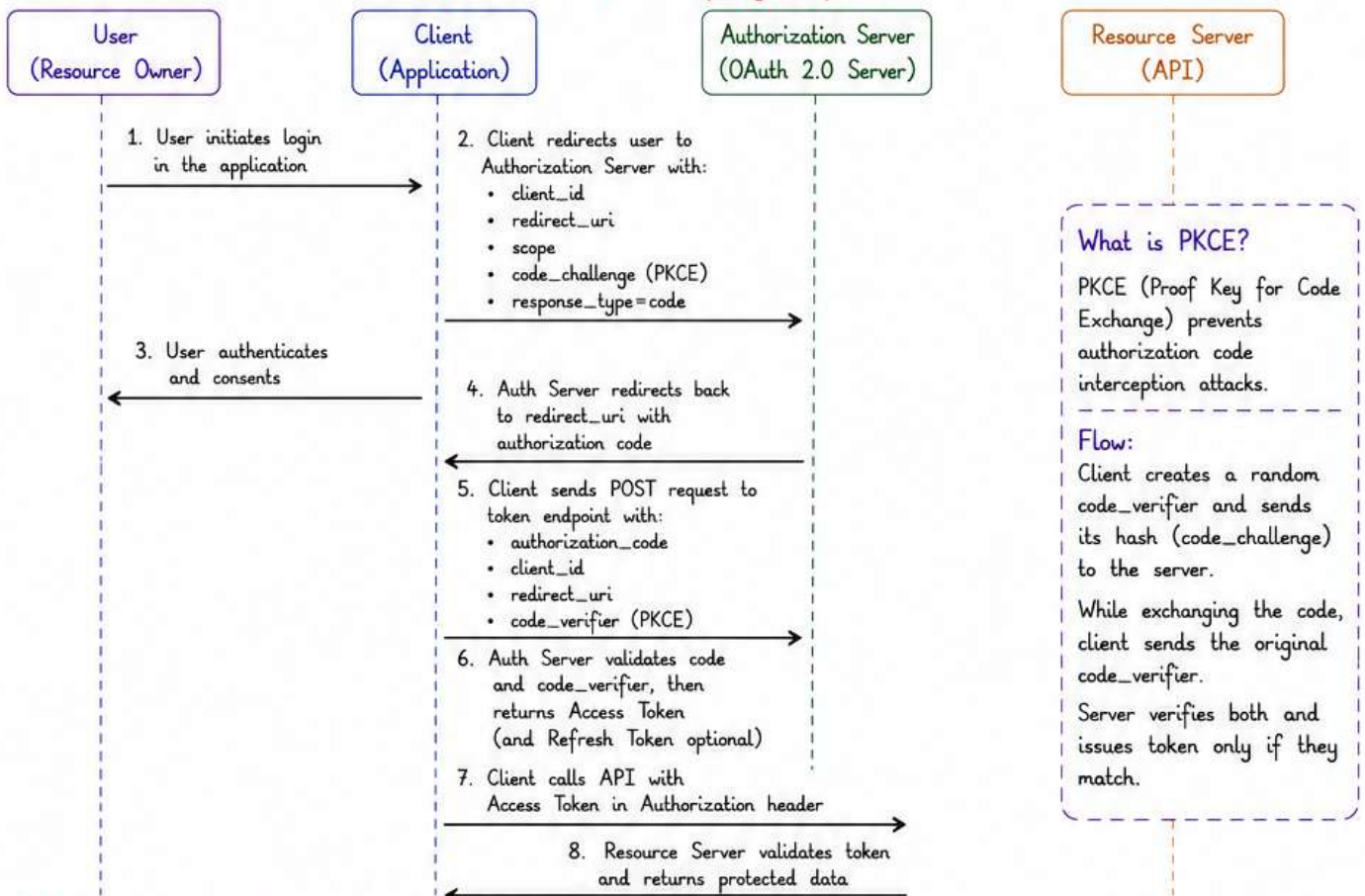
PAGE 8

Authorization Code Flow is the most secure OAuth 2.0 flow for web and mobile apps. PKCE adds extra protection for public clients.

### 8.1 Actors Involved



### 8.2 Authorization Code Flow (With PKCE) - Step by Step



### 8.3 Advantages of Authorization Code Flow

- ✓ Most secure flow for web and mobile apps.
- ✓ Authorization code is short lived.
- ✓ PKCE protects against code interception.
- ✓ Access token is never exposed in browser URL.
- ✓ Works well with Confidential and Public clients.

### 8.4 When to Use?

- Web applications
- Mobile applications
- Applications that can securely store client secret (Confidential clients)
- When you need the highest level of security



**TIP:**

Always use Authorization Code Flow with PKCE for modern applications. It is the recommended and most secure way.

IG: abhi\_techhub



## 9. OAUTH vs JWT vs SESSIONS

PAGE 9

They solve different problems. Choose the right tool for the right job.

| Feature               | OAuth 2.0                                              | JWT                                              | Sessions                               |
|-----------------------|--------------------------------------------------------|--------------------------------------------------|----------------------------------------|
| What it is            | Authorization framework (Delegated Access)             | Token format (For Authentication)                | Server-side session management         |
| Main Purpose          | Allow a client to access resources on behalf of a user | Prove the identity of the user and carry claims  | Maintain user state on the server      |
| Uses Tokens?          | Yes (Access Token) + Refresh Token                     | Yes (Access Token) + Refresh Token               | No (Uses Session ID in Cookie)         |
| Where is Data Stored? | Authorization Server (+ Client)                        | Inside the token (Stateless)                     | On Server (Session Store)              |
| State                 | Stateful (Auth Server) but supports Stateless          | Stateless (Self-contained)                       | Stateful (Server-side)                 |
| Scalability           | High (Designed for scale)                              | Very High (No server state needed)               | Low to Medium (Session store needed)   |
| Revocation            | Possible (Token revocation, introspection)             | Hard (Need blacklist or short expiry)            | Easy (Delete session from server)      |
| Best For              | 3rd party access, SSO, API access delegation           | Microservices, Mobile apps, SPAs, Stateless APIs | Traditional web apps, internal systems |
| Delivered By          | Authorization Server (e.g., Google, Auth0)             | Your Application / Auth Service                  | Your Application (Server)              |

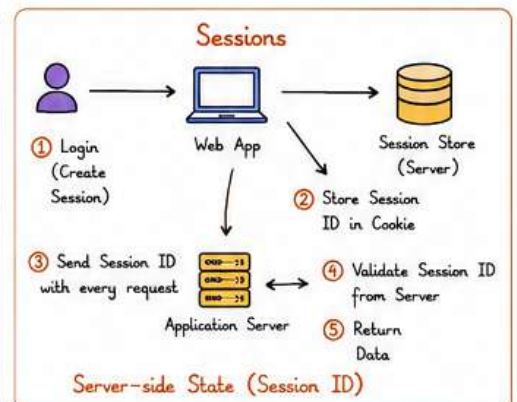
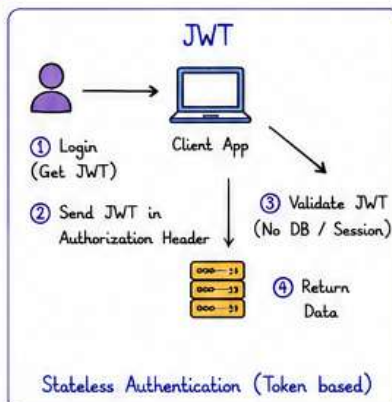
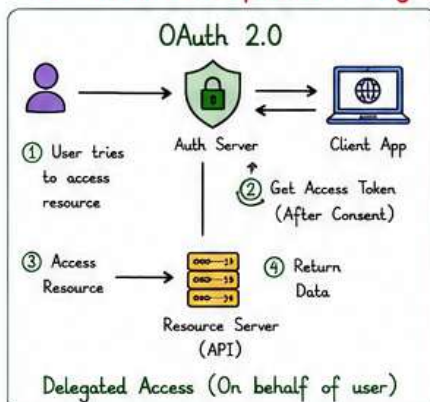
### 9.1 Common Misconceptions (Clear it!)

- ❌ OAuth 2.0 is NOT an authentication protocol. It is an authorization framework.
- ❌ JWT is NOT OAuth. It is just a token format.
- ❌ OAuth 2.0 can use JWT as Access Token, but it can also use opaque tokens.
- ✓ Sessions can be used with OAuth 2.0 (e.g., in server-side web apps).
- ✓ JWT can be used without OAuth 2.0 (for first-party authentication).

### 9.2 When to Use What?

- Use OAuth 2.0 When:
  - You want to allow access to user data by another app.
  - You are building login with Google / Facebook / or any third-party app.
- Use JWT When:
  - You need stateless authentication for APIs.
  - You are building microservices or mobile / SPA apps.
- Use Sessions When:
  - You are building a traditional web application.
  - You want easy logout and server-side control.

### 9.3 Quick Comparison Diagram



**TIP:** Remember the rule of thumb:

OAuth 2.0 = Authorization | JWT = Authentication (Token) | Sessions = Server State

IG: abhi\_techhub ❤️

## 10. SECURITY BEST PRACTICES

PAGE 10

Follow these best practices to keep your authentication secure and robust.

### 1. Use HTTPS

- Always use HTTPS to protect tokens and data in transit.
- Never send tokens over HTTP.



### 2. Short Lived Access Token

- Keep Access Token short lived (5 - 15 min).
- Use Refresh Token for long term access.



### 3. Secure Storage

- Store tokens in HttpOnly, Secure, SameSite cookies.
- Avoid storing tokens in LocalStorage or SessionStorage.



### 4. Validate Tokens

- Always verify token signature and expiry.
- Validate issuer (iss), audience (aud) and other claims.
- Reject invalid or tampered tokens.



### 5. Refresh Token Security

- Rotate Refresh Token after every use.
- Revoke old tokens immediately.
- Set proper expiry for Refresh Token.



### 6. Least Privilege

- Grant only the permissions that are required.
- Follow the principle of least privilege.



### 7. Handle Logout Properly

- Clear tokens from client and server.
- Invalidate Refresh Token on logout.
- Destroy session (if using sessions).



### 8. Protect Against CSRF

- Use SameSite cookies.
- Use CSRF tokens for state changing operations.



### 9. Monitor & Log

- Log login/logout attempts.
- Monitor suspicious activities.
- Alert on unusual behavior.



### 10. Keep Secrets Safe

- Never expose your client secret.
- Store secrets in environment variables.



### 10.1 Token Security Checklist

- ☒ Use HTTPS everywhere.
- ☒ Access Token is short lived (5 - 15 min).
- ☒ Refresh Token is long lived and rotated.
- ☒ Tokens are stored in HttpOnly, Secure, SameSite cookies.
- ☒ Tokens are not stored in LocalStorage / SessionStorage.
- ☒ Token signature and expiry are validated.
- ☒ Implement proper logout and token revocation.
- ☒ Protect against CSRF attacks.
- ☒ Use least privilege when granting scopes/permissions.
- ☒ Monitor and log authentication events.
- ☒ Keep secrets safe and never commit them.

### 10.2 Common Security Threats

| Threat              | How to Prevent                                                         |
|---------------------|------------------------------------------------------------------------|
| Token Theft (XSS)   | Use HttpOnly cookies, Content Security Policy (CSP), input validation. |
| Replay Attack       | Use short lived tokens, token rotation, unique jti claim.              |
| Token Tampering     | Validate signature, use strong algorithms (HS256 / RS256 / ES256).     |
| CSRF Attack         | Use SameSite cookies, CSRF tokens.                                     |
| Brute Force Login   | Rate limiting, account lockout, CAPTCHA.                               |
| Exposed Secrets     | Use environment variables, secret management tools.                    |
| Expired Token Usage | Validate expiry and return proper error (401 Unauthorized).            |

### 10.3 Quick Tips

- Security is not optional, it is essential.
- A small mistake in authentication can lead to big breaches.
- Keep your tokens safe, validate always, trust never.
- Follow OWASP guidelines for secure authentication.

### 10.4 Tools You Can Use

- JWT Debugger → [jwt.io](https://jwt.io)
- OAuth Playground → [oauth.net](https://oauth.net) / Auth0 Playground
- Postman → For testing OAuth flows
- OWASP → For security best practices



**TIP:** Security is a process, not a product.  
Keep learning, keep implementing, keep securing!

IG: [abhi\\_techhub](https://www.instagram.com/abhi_techhub)



## 11. COMMON INTERVIEW QUESTIONS

Important questions asked in interviews with crisp answers.

1. What is the difference between Authentication and Authorization?
    - Authentication verifies who you are.
    - Authorization decides what you can access.
  2. What is JWT?
    - JWT (JSON Web Token) is a compact, URL-safe token used to securely transmit information between parties.
  3. What are the parts of a JWT?
    - Header . Payload . Signature  
(Encoded and separated by dots.)
  4. What is the use of Refresh Token?
    - Refresh Token is used to get a new Access Token when the old one expires.
  5. What is the difference between OAuth 2.0 and JWT?
    - OAuth 2.0 is an authorization framework.
    - JWT is a token format used inside OAuth 2.0.
  6. What is scope in OAuth 2.0?
    - Scope defines the level of access the client is requesting (e.g., read:profile, write:email).
  7. What is PKCE in OAuth 2.0?
    - PKCE (Proof Key for Code Exchange) is an extension used to prevent authorization code interception attacks.
  8. How is JWT different from Sessions?
    - JWT is stateless and stored on client.
    - Sessions are stateful and stored on server.
  9. Where should JWT be stored?
    - Best: HttpOnly, Secure, SameSite cookies.
    - Avoid: localStorage and sessionStorage.
  10. How do you revoke a JWT?
    - JWT cannot be revoked directly.
    - Use short expiry + Refresh Token + token blacklist.

## 12. JWT Example (Decoded)

Token:

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.  
eyJzdWIiOiIxMjMjLjJ1YW1lIjoiQWJoaSIs  
InJvbGUiOiJ1c2V5IiwiaWF0IjoxNzI1NjMxMjAw  
.  
dF8JxkKQ3Y8Q2F7wqQZ3aJ7K2LJQx9J0b4V7k

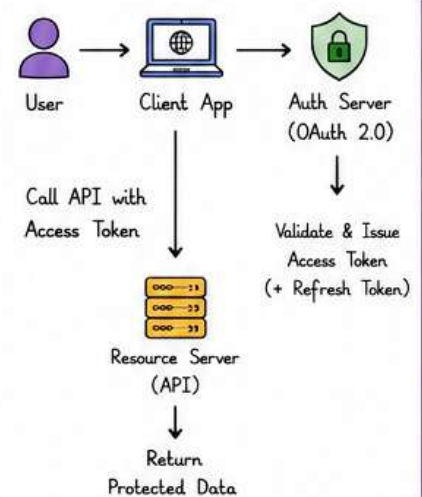
Decoded:

|           |                                                                                      |
|-----------|--------------------------------------------------------------------------------------|
| Header    | { "alg": "HS256", "typ": "JWT" }                                                     |
| Payload   | { "sub": "123",<br>"name": "Abhi", "role": "user",<br>"iat": 1715631200 }            |
| Signature | HMACSHA256(<br>base64UrlEncode(header) + "." +<br>base64UrlEncode(payload), secret ) |

### 13. OAuth 2.0 Grants

|                                                                                     |                                          |                                                                    |
|-------------------------------------------------------------------------------------|------------------------------------------|--------------------------------------------------------------------|
|  | <p>Authorization<br/>Code Grant</p>      | <p>For web apps.<br/>Most secure.<br/>Uses PKCE.</p>               |
|  | <p>Implicit Grant</p>                    | <p>For SPAs (legacy).<br/>Token in URL<br/>fragment.</p>           |
|  | <p>Resource Owner<br/>Password Grant</p> | <p>User gives<br/>username &amp; password<br/>to app directly.</p> |
|  | <p>Client Credentials<br/>Grant</p>      | <p>For machine to<br/>machine (no user<br/>involved).</p>          |
|  | <p>Refresh Token<br/>Grant</p>           | <p>Used to get new<br/>Access Token when<br/>old one expires.</p>  |

#### 14. Token Flow Summary



## 15. Common Mistakes to Avoid

- ✗ Storing JWT in localStorage.
- ✗ Using weak or missing token expiry.
- ✗ Not using HTTPS.
- ✗ Not validating token signature.
- ✗ Giving too many scopes to clients.
- ✗ Exposing client secret in frontend.
- ✗ Not rotating Refresh Token.
- ✗ Not handling token revocation.

## 16. Pro Tips

- ✓ Keep Access Token short-lived.
- ✓ Use Refresh Token rotation.
- ✓ Use HttpOnly, Secure, SameSite cookies.
- ✓ Validate everything on server.
- ✓ Use PKCE for public clients.
- ✓ Monitor and log auth events.
- ✓ Implement rate limiting on login.
- ✓ Follow OWASP recommendations.

### 17. Remember This



- OAuth 2.0 = Authorization Framework
- JWT = Token Format
- Sessions = Server State

Choose the right tool for the right problem!



**TIP:** Strong authentication is the first line of defense.  
Always think like an attacker and build like a defender!

IG: [abhi\\_techhub](#)



## 12. REAL WORLD EXAMPLES

PAGE 12

Let's see how popular platforms use OAuth 2.0 and JWT.

| Platform / Service                                                                                     | Uses OAuth 2.0? | Uses JWT? | How They Use It                                                                                                                                 |
|--------------------------------------------------------------------------------------------------------|-----------------|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------|
|  Google               | Yes             | Yes       | <ul style="list-style-type: none"> <li>• OAuth 2.0 for login with Google.</li> <li>• JWT (ID Token) to identify the user.</li> </ul>            |
|  Facebook             | Yes             | Yes       | <ul style="list-style-type: none"> <li>• OAuth 2.0 for Facebook Login.</li> <li>• JWT to securely send user info to app.</li> </ul>             |
|  GitHub               | Yes             | Sometimes | <ul style="list-style-type: none"> <li>• OAuth 2.0 for third-party apps.</li> <li>• JWT for GitHub Apps (identification).</li> </ul>            |
|  Twitter (X)          | Yes             | Yes       | <ul style="list-style-type: none"> <li>• OAuth 2.0 for user login.</li> <li>• JWT for access tokens in API calls.</li> </ul>                    |
|  Microsoft (Azure AD) | Yes             | Yes       | <ul style="list-style-type: none"> <li>• OAuth 2.0 / OpenID Connect for login.</li> <li>• JWT (ID Token) for user identity.</li> </ul>          |
|  Stripe               | Yes             | No        | <ul style="list-style-type: none"> <li>• OAuth 2.0 for API authentication.</li> <li>• Uses API keys &amp; tokens (not JWT).</li> </ul>          |
|  Your App            | Yes             | Yes       | <ul style="list-style-type: none"> <li>• OAuth 2.0 for login (Google, Facebook, etc.)</li> <li>• JWT for managing sessions and APIs.</li> </ul> |

### 12.1 Design Decision Guide

Which one should you use?

| Use OAuth 2.0 when...                                                                                                                                                                                                                                                        | Use JWT when...                                                                                                                                                                                                                                                       |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>• You want users to login with Google, Facebook, etc.</li> <li>• You are giving third-party access to your API.</li> <li>• You need delegated access (with scopes).</li> <li>• You need a standards-based and secure flow.</li> </ul> | <ul style="list-style-type: none"> <li>• You need to identify the user after login.</li> <li>• You want stateless and fast authentication.</li> <li>• You are building your own auth system.</li> <li>• You need to send claims between services securely.</li> </ul> |

### 12.2 Summary Cheat Sheet

| OAuth 2.0                                                                                                                                                                                                                                                  | JWT                                                                                                                                                                                                           |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>✓ Authorization Framework</li> <li>✓ Delegated Access</li> <li>✓ Uses Access Token</li> <li>✓ Multiple Flows</li> <li>✓ Stateful (AS stores info)</li> <li>✓ For Login with 3rd Party (Google, FB, etc.)</li> </ul> | <ul style="list-style-type: none"> <li>✓ Token Format</li> <li>✓ Authentication</li> <li>✓ Contains Claims</li> <li>✓ Self-contained</li> <li>✓ Stateless</li> <li>✓ For your own API and services</li> </ul> |

### 12.3 OAuth 2.0 vs JWT vs Sessions (At a Glance)

| Feature       | OAuth 2.0                                   | JWT                                 | Sessions                  |
|---------------|---------------------------------------------|-------------------------------------|---------------------------|
| Type          | Authorization Framework                     | Token Format                        | Server-side State         |
| Purpose       | Allow access to resources on behalf of user | Prove identity & carry claims       | Maintain user state       |
| Token / ID    | Access Token (and Refresh Token)            | JWT                                 | Session ID (in Cookie)    |
| Stored Where? | Auth Server                                 | Client (self-contained)             | Server (Session Store)    |
| State         | Stateful (AS keeps info)                    | Stateless                           | Stateful                  |
| Best For      | 3rd Party Login, API Access Delegation      | APIs, Microservices, Mobile Apps    | Traditional Web Apps      |
| Revocation    | Easy (revoke token)                         | Hard (use short expiry + blacklist) | Easy (delete session)     |
| Example       | Login with Google                           | Access your own API                 | E-commerce cart, web apps |



**TIP:**

- ✓ OAuth 2.0 = Get Permission
- ✓ JWT = Prove Identity
- ✓ Sessions = Maintain State

Understand the difference.  
Choose the right tool.  
Build secure, scalable apps!

IG: abhi\_techhub

